Database Locks: Comprehensive Guide

Table of Contents

1. Introduction
2. Lock Types
3. Pessimistic Locking
4. Shared Locking
5. How Locks Work
6. Benefits
7. MySQL Examples
8. MSSQL Examples
9. Transaction Examples
10. Deadlock Prevention
11. Deadlock Detection

Introduction

Database locks are mechanisms used to control concurrent access to data. They prevent multiple transactions from interfering with each other and ensure data consistency. Understanding locks is critical for building scalable, reliable applications.

When are locks needed?

- Multiple users accessing the same data simultaneously
- Preventing dirty reads, non-repeatable reads, and phantom reads
- Maintaining ACID (Atomicity, Consistency, Isolation, Durability) properties

Lock Types

Exclusive Locks (Write Locks)

- Prevent other transactions from reading OR writing the resource
- Only one transaction can hold an exclusive lock on a resource at a time
- Used when modifying data (INSERT, UPDATE, DELETE)

Shared Locks (Read Locks)

- Allow multiple transactions to read the same resource simultaneously
- Prevent other transactions from acquiring exclusive locks
- Used when reading data with locking requirements

Intent Locks

- Used at table level to indicate that transaction intends to acquire locks at row/page level
- Improves lock management efficiency

Pessimistic Locking

Definition

Pessimistic locking assumes conflicts WILL occur and locks resources upfront before any modifications. The transaction acquires locks immediately to prevent other transactions from interfering.

Characteristics

- **Proactive approach**: Lock first, act later
- **Resource blocking**: Resources remain locked for the duration of the transaction
- **Deadlock risk**: Higher potential for deadlocks
- **Performance**: Can reduce concurrency but prevents conflicts
- **Predictable behavior**: Serializes access to locked resources

When to Use

- High contention scenarios (many transactions competing for same resources)
- Critical business operations where conflicts are expensive
- Short transactions
- When optimistic locking has failed repeatedly

Disadvantages

- Can significantly reduce concurrency
- Higher deadlock probability
- Resources locked longer than necessary
- Blocked transactions create bottlenecks

Shared Locking

Definition

Shared locks (S-locks or read locks) allow multiple transactions to read the same resource without being able to modify it. Once acquired, other transactions cannot get exclusive locks on that resource.

Characteristics

- **Non-exclusive**: Multiple transactions can hold shared locks simultaneously
- **Read-only**: Prevents modifications by other transactions
- **Compatibility**: Compatible with other shared locks, incompatible with exclusive locks
- **Duration**: Usually held until transaction completes

Lock Compatibility Matrix

Lock Type	Exclusive	Shared	Intent Exclusive	Intent Shared
Exclusive	❌	❌	❌	❌
Shared	❌	✅	❌	✅
Intent Exclusive	❌	❌	❌	❌

Lock Type	Exclusive	Shared	Intent Exclusive	Intent Shared
Intent Shared	❌	✅	❌	✅

When to Use

- Ensuring read consistency without allowing modifications
- Preventing other transactions from modifying data while you're reading
- Implementing repeatable read isolation
- Long-running reports or analytics queries

How Locks Work

Lock Granularity

Row-Level Locks

- Most fine-grained locking
- Locks individual rows
- Highest concurrency, highest overhead
- Best for OLTP systems

Page-Level Locks

- Locks 8KB pages (in most DBMS)
- Compromise between row and table locks
- Medium overhead and concurrency
- Automatic escalation possible

Table-Level Locks

- Locks entire table
- Lowest concurrency, lowest overhead
- Used when whole-table operations occur
- Lock escalation destination

Lock Acquisition Process

```
Transaction 1              Lock Manager           Resource
    |                          |                      |
    |--Request Lock---------->|                      |
    |                          |--Check Compatibility |
    |                          |<--Available----------|
    |<--Grant Lock------------|                      |
    |                          |                      |
    |--Read/Modify Data------->|                      |
    |                          |--Lock Held-----------|
    |                          |                      |
    |--Commit/Rollback-------->|                      |
    |                          |--Release Lock--------|
```

Lock Wait Queue

When a lock cannot be granted:

1. Transaction enters wait queue
2. Lock manager suspends transaction
3. When lock becomes available, manager grants it
4. Transaction resumes execution

Benefits

Data Consistency

- Ensures ACID properties
- Prevents dirty reads, non-repeatable reads, phantom reads
- Maintains referential integrity

Prevents Race Conditions

- Multiple transactions can safely access same resource
- No lost updates
- No concurrent modification conflicts

Isolation

- Transactions appear to execute serially
- One transaction's work doesn't interfere with others
- Predictable behavior

Reduced Application Complexity

- Database handles synchronization
- No need for application-level locking mechanisms
- Cleaner, more maintainable code

MySQL Examples

Basic Pessimistic Locking with FOR UPDATE

```sql
-- Session 1: Read and lock a row
START TRANSACTION;
SELECT * FROM accounts
WHERE account_id = 1
FOR UPDATE;  -- Acquires exclusive lock

-- Do business logic here
UPDATE accounts
SET balance = balance - 100
WHERE account_id = 1;

COMMIT;  -- Lock is released
```

Shared Lock Example

```sql
-- Session 1: Acquire shared lock for reading
START TRANSACTION;
SELECT * FROM accounts
WHERE account_id = 1
FOR SHARE;  -- Acquires shared lock (MySQL 8.0+)

-- Read data, multiple transactions can do this simultaneously
SELECT * FROM accounts WHERE account_id = 1;

COMMIT;
```

Multiple Row Locking

```sql
-- Lock multiple rows for update
START TRANSACTION;
SELECT * FROM accounts
WHERE customer_id = 100
FOR UPDATE;  -- Locks all matching rows

UPDATE accounts
SET status = 'locked'
WHERE customer_id = 100;

COMMIT;
```

Money Transfer Example (Pessimistic)

```sql
-- Transferring money between accounts
DELIMITER //
CREATE PROCEDURE transfer_money(
    IN from_account INT,
    IN to_account INT,
    IN amount DECIMAL(10,2)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Transaction failed' as status;
    END;

    START TRANSACTION;

    -- Lock both accounts (always in same order to prevent deadlock)
    SELECT balance INTO @from_balance
    FROM accounts
    WHERE account_id = from_account
    FOR UPDATE;

    SELECT balance INTO @to_balance
    FROM accounts
    WHERE account_id = to_account
    FOR UPDATE;

    -- Validate
    IF @from_balance < amount THEN
        ROLLBACK;
        SELECT 'Insufficient funds' as status;
    ELSE
        UPDATE accounts
        SET balance = balance - amount
        WHERE account_id = from_account;

        UPDATE accounts
        SET balance = balance + amount
        WHERE account_id = to_account;

        COMMIT;
```

```
        SELECT 'Transfer successful' as status;
    END IF;
END //
DELIMITER ;

-- Call the procedure
CALL transfer_money(1, 2, 50.00);
```

Nowait and Skip Locked

```
-- MySQL 8.0.1+ NOWAIT: Fail immediately if lock can't be acquired
SELECT * FROM accounts
WHERE account_id = 1
FOR UPDATE NOWAIT;

-- MySQL 8.0.1+ SKIP LOCKED: Skip locked rows
SELECT * FROM accounts
WHERE status = 'active'
FOR UPDATE SKIP LOCKED
LIMIT 5;  -- Get first 5 unlocked active accounts
```

Checking Locks

```
-- View current locks in MySQL 8.0+
SELECT * FROM performance_schema.data_locks;

-- View lock wait information
SELECT * FROM performance_schema.data_lock_waits;

-- Older MySQL versions
SHOW PROCESSLIST;
SHOW OPEN TABLES WHERE In_use > 0;
```

MSSQL Examples

Basic Pessimistic Locking with XLOCK

```sql
-- Read and lock a row with exclusive lock
BEGIN TRANSACTION;

SELECT * FROM accounts (XLOCK)
WHERE account_id = 1;

-- Do business logic
UPDATE accounts
SET balance = balance - 100
WHERE account_id = 1;

COMMIT;  -- Lock released
```

Shared Lock Example

```sql
-- Acquire shared lock for reading
BEGIN TRANSACTION;

SELECT * FROM accounts (HOLDLOCK)
WHERE account_id = 1;

-- Other transactions can also read but not modify
SELECT * FROM accounts WHERE account_id = 1;

COMMIT;
```

Lock Hints in MSSQL

```sql
-- XLOCK: Exclusive lock (write)
SELECT * FROM accounts (XLOCK) WHERE id = 1;

-- SLOCK: Shared lock (read)
SELECT * FROM accounts (SLOCK) WHERE id = 1;

-- HOLDLOCK: Retain shared lock until transaction ends
SELECT * FROM accounts (HOLDLOCK, SLOCK) WHERE id = 1;

-- NOLOCK: Read uncommitted (dirty read - dangerous!)
SELECT * FROM accounts (NOLOCK) WHERE id = 1;

-- PAGLOCK: Page-level lock instead of row-level
SELECT * FROM accounts (XLOCK, PAGLOCK) WHERE id = 1;

-- TABLOCK: Table-level lock
SELECT * FROM accounts (XLOCK, TABLOCK) WHERE id = 1;

-- UPDLOCK: Update lock (can be upgraded to exclusive)
SELECT * FROM accounts (UPDLOCK) WHERE id = 1;
```

Money Transfer Example (MSSQL)

```sql
CREATE PROCEDURE sp_transfer_money
    @FromAccount INT,
    @ToAccount INT,
    @Amount DECIMAL(10,2)
AS
BEGIN
    SET XACT_ABORT ON;  -- Automatic rollback on error

    BEGIN TRANSACTION;

    -- Lock both accounts in consistent order (smaller ID first)
    DECLARE @LockAccount1 INT, @LockAccount2 INT;

    SET @LockAccount1 = CASE
        WHEN @FromAccount < @ToAccount THEN @FromAccount
        ELSE @ToAccount
    END;

    SET @LockAccount2 = CASE
        WHEN @FromAccount < @ToAccount THEN @ToAccount
        ELSE @FromAccount
    END;

    -- Lock first account
    DECLARE @Balance1 DECIMAL(10,2);
    SELECT @Balance1 = balance
    FROM accounts (XLOCK)
    WHERE account_id = @LockAccount1;

    -- Lock second account
    DECLARE @Balance2 DECIMAL(10,2);
    SELECT @Balance2 = balance
    FROM accounts (XLOCK)
    WHERE account_id = @LockAccount2;

    -- Validate
    IF (@FromAccount = @LockAccount1 AND @Balance1 < @Amount) OR
       (@FromAccount = @LockAccount2 AND @Balance2 < @Amount)
    BEGIN
        ROLLBACK;
        THROW 50001, 'Insufficient funds', 1;
```

```sql
    END;

    -- Perform transfer
    UPDATE accounts
    SET balance = balance - @Amount
    WHERE account_id = @FromAccount;

    UPDATE accounts
    SET balance = balance + @Amount
    WHERE account_id = @ToAccount;

    COMMIT;
    RETURN 0;
END;

-- Execute
EXEC sp_transfer_money @FromAccount = 1, @ToAccount = 2, @Amount = 50.00;
```

Row Constructor for Multiple Locks

```sql
-- Lock multiple rows efficiently
BEGIN TRANSACTION;

SELECT account_id, balance
FROM accounts
WHERE account_id IN (10, 20, 30)
FOR UPDATE;  -- or (XLOCK) in older versions

-- Process all locked accounts
UPDATE accounts
SET balance = balance * 1.01
WHERE account_id IN (10, 20, 30);

COMMIT;
```

Checking Locks in MSSQL

```sql
-- View current locks
SELECT * FROM sys.dm_tran_locks;

-- View lock waits
SELECT * FROM sys.dm_os_waiting_tasks;

-- Blocking information
EXEC sp_who2 'ACTIVE';

-- More detailed blocking
SELECT blocking_session_id, session_id, wait_type
FROM sys.dm_exec_requests
WHERE blocking_session_id > 0;
```

Transaction Examples

Example 1: Inventory Update (Pessimistic)

```sql
-- MySQL
START TRANSACTION;

-- Lock inventory row
SELECT quantity
FROM inventory
WHERE product_id = 100
FOR UPDATE;

-- Check if available
IF quantity >= 10 THEN
    UPDATE inventory
    SET quantity = quantity - 10
    WHERE product_id = 100;
    COMMIT;
ELSE
    ROLLBACK;
END IF;
```

```sql
-- MSSQL
BEGIN TRANSACTION;

DECLARE @Qty INT;
SELECT @Qty = quantity
FROM inventory (XLOCK)
WHERE product_id = 100;

IF @Qty >= 10
BEGIN
    UPDATE inventory
    SET quantity = quantity - 10
    WHERE product_id = 100;
    COMMIT TRANSACTION;
END
ELSE
BEGIN
    ROLLBACK TRANSACTION;
END
```

Example 2: Vacation Balance Deduction

```sql
-- MySQL: Book vacation days
START TRANSACTION;

-- Lock employee record
SELECT vacation_days
FROM employees
WHERE employee_id = 5
FOR UPDATE;

-- Deduct days and log
UPDATE employees
SET vacation_days = vacation_days - 5
WHERE employee_id = 5;

INSERT INTO vacation_log (employee_id, days, from_date, to_date)
VALUES (5, 5, CURDATE(), CURDATE() + INTERVAL 5 DAY);

COMMIT;
```

Example 3: Batch Processing with Locks

```sql
-- MySQL: Process pending orders
START TRANSACTION;

-- Get 10 pending orders and lock them
SELECT order_id, total
FROM orders
WHERE status = 'pending'
FOR UPDATE
LIMIT 10;

-- Update status
UPDATE orders
SET status = 'processing', updated_at = NOW()
WHERE status = 'pending'
LIMIT 10;

-- Log the processing
INSERT INTO processing_log (processed_count, timestamp)
VALUES (10, NOW());

COMMIT;
```

Example 4: Shared Lock with Read Consistency

```sql
-- MSSQL: Generate report with consistent data
BEGIN TRANSACTION;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Lock data but allow other reads
SELECT department, COUNT(*) as employee_count, AVG(salary) as avg_salary
FROM employees (HOLDLOCK, SLOCK)
GROUP BY department;

-- This read is guaranteed to show same data
SELECT * FROM employees (HOLDLOCK, SLOCK)
WHERE salary > 100000;

COMMIT;
```

Deadlock Prevention

Deadlock Basics

A deadlock occurs when:

1. Transaction A holds lock on Resource 1, waits for Resource 2
2. Transaction B holds lock on Resource 2, waits for Resource 1
3. Both transactions are blocked indefinitely

Prevention Strategies

1. Lock Ordering (Most Important)

Always acquire locks in the same order across all transactions.

```sql
-- CORRECT: Always lock in order (account_id ascending)
-- MySQL
START TRANSACTION;
SELECT * FROM accounts WHERE account_id = 1 FOR UPDATE;
SELECT * FROM accounts WHERE account_id = 2 FOR UPDATE;
COMMIT;

-- MSSQL
BEGIN TRANSACTION;
SELECT * FROM accounts (XLOCK) WHERE account_id = 1;
SELECT * FROM accounts (XLOCK) WHERE account_id = 2;
COMMIT;

-- WRONG: Inconsistent order causes deadlock
-- Transaction 1: Lock 1, then Lock 2
-- Transaction 2: Lock 2, then Lock 1  <- DEADLOCK!
```

2. Minimize Lock Duration

```sql
-- WRONG: Lock held too long
START TRANSACTION;
SELECT * FROM accounts WHERE id IN (1, 2) FOR UPDATE;
-- Do lengthy external API call here
-- ... 30 seconds of processing ...
COMMIT;

-- CORRECT: Minimize locked section
SELECT * FROM accounts WHERE id IN (1, 2);  -- No lock
-- Do lengthy external API call
-- ... 30 seconds of processing ...

START TRANSACTION;
SELECT * FROM accounts WHERE id IN (1, 2) FOR UPDATE;
UPDATE accounts SET status = 'processed' WHERE id IN (1, 2);
COMMIT;
```

3. Use NOWAIT or SKIP LOCKED

```sql
-- MySQL 8.0+
START TRANSACTION;
SELECT * FROM accounts
WHERE account_id = 1
FOR UPDATE NOWAIT;  -- Fail immediately if locked
-- OR
FOR UPDATE SKIP LOCKED;  -- Skip if locked, continue with others
COMMIT;

-- MSSQL with TIMEOUT
SET DEADLOCK_PRIORITY HIGH;
-- Try operation, will fail faster if deadlock detected
```

4. Consistent Isolation Levels

```sql
-- Set appropriate isolation level
-- MySQL
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
-- or REPEATABLE READ (default)
-- or SERIALIZABLE

-- MSSQL
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
-- or REPEATABLE READ
-- or SERIALIZABLE
```

5. Keep Transactions Short

```sql
-- WRONG: Long transaction
START TRANSACTION;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
PERFORM_LENGTHY_OPERATION();  -- 10 seconds
UPDATE accounts SET ... WHERE id = 1;
COMMIT;

-- CORRECT: Short transaction
PERFORM_LENGTHY_OPERATION();
START TRANSACTION;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
UPDATE accounts SET ... WHERE id = 1;
COMMIT;
```

6. Batch Similar Operations

```sql
-- Process all pending orders together to minimize lock contention
START TRANSACTION;

SELECT order_id FROM orders
WHERE status = 'pending'
FOR UPDATE;

UPDATE orders
SET status = 'processing'
WHERE status = 'pending';

COMMIT;
```

Deadlock Prevention Checklist

- ✅ Always acquire locks in consistent order
- ✅ Keep transactions as short as possible
- ✅ Avoid user input during transactions
- ✅ Avoid external API calls while holding locks
- ✅ Use appropriate isolation levels
- ✅ Test for deadlocks under load
- ✅ Use connection pooling with timeout
- ✅ Consider optimistic locking for high-contention scenarios

Deadlock Detection

MySQL Deadlock Detection

View Deadlock Information

```sql
-- Enable InnoDB status logging
SET GLOBAL innodb_print_all_deadlocks = ON;

-- Check error log for deadlock details
-- Linux: tail -f /var/log/mysql/error.log

-- Get detailed lock information (MySQL 8.0+)
SELECT * FROM performance_schema.data_locks;

SELECT * FROM performance_schema.data_lock_waits;
```

Deadlock Example in MySQL

```sql
-- Session 1:
START TRANSACTION;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
-- Wait here...
SELECT * FROM accounts WHERE id = 2 FOR UPDATE;

-- Session 2 (in parallel):
START TRANSACTION;
SELECT * FROM accounts WHERE id = 2 FOR UPDATE;
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;  -- DEADLOCK!

-- Error: ERROR 1213 (40001): Deadlock found when trying to get lock
```

Handling Deadlock in Application (MySQL)

```java
// Java example with retry logic
public void transferMoney(int fromId, int toId, double amount)
    throws SQLException {

    int maxRetries = 3;
    int retryCount = 0;

    while (retryCount < maxRetries) {
        try {
            connection.setAutoCommit(false);

            // Lock first account
            String sql1 = "SELECT * FROM accounts WHERE id = ? FOR UPDATE";
            PreparedStatement ps1 = connection.prepareStatement(sql1);
            ps1.setInt(1, fromId);
            ps1.executeQuery();

            // Lock second account
            PreparedStatement ps2 = connection.prepareStatement(sql1);
            ps2.setInt(1, toId);
            ps2.executeQuery();

            // Perform transfer
            // ... update logic ...

            connection.commit();
            return;

        } catch (SQLException e) {
            if ("40001".equals(e.getSQLState())) {
                // Deadlock detected, retry
                connection.rollback();
                retryCount++;
                Thread.sleep(100 * retryCount);  // Exponential backoff
            } else {
                throw e;
            }
        }
    }
```

```
    throw new SQLException("Transfer failed after " + maxRetries + " retries");
}
```

MSSQL Deadlock Detection

View Deadlock Graph

```sql
-- Enable deadlock graph (requires profiler or Extended Events)
-- In SQL Server Management Studio:
-- Tools > SQL Server Profiler > Trace Properties
-- Events: Locks > Deadlock graph

-- Or use Extended Events (SQL Server 2008+)
CREATE EVENT SESSION deadlock_events ON SERVER
ADD EVENT sqlserver.xml_deadlock_report
ADD TARGET package0.event_file(
    SET filename=N'C:\DeadlockLog.xel')
WITH (STARTUP_STATE=ON);

-- View recent deadlocks
SELECT * FROM sys.dm_exec_requests WHERE blocking_session_id > 0;
```

Deadlock Example in MSSQL

```sql
-- Session 1:
BEGIN TRANSACTION;
SELECT * FROM accounts (XLOCK) WHERE id = 1;
-- Wait here...
SELECT * FROM accounts (XLOCK) WHERE id = 2;

-- Session 2 (in parallel):
BEGIN TRANSACTION;
SELECT * FROM accounts (XLOCK) WHERE id = 2;
SELECT * FROM accounts (XLOCK) WHERE id = 1;  -- DEADLOCK!

-- Error: Msg 1205, Level 13, State 45
-- Transaction (Process ID 54) was deadlocked
```

Handling Deadlock in Application (MSSQL)

```
// C# example with retry logic
public void TransferMoney(int fromId, int toId, decimal amount)
{
    const int maxRetries = 3;
    int retryCount = 0;

    while (retryCount < maxRetries)
    {
        try
        {
            using (var connection = new SqlConnection(connectionString))
            {
                connection.Open();

                using (var transaction = connection.BeginTransaction())
                {
                    // Lock accounts
                    var cmd = connection.CreateCommand();
                    cmd.Transaction = transaction;

                    cmd.CommandText = @"
                        SELECT balance FROM accounts (XLOCK)
                        WHERE id = @id1
                    ";
                    cmd.Parameters.AddWithValue("@id1", fromId);
                    cmd.ExecuteScalar();

                    cmd.CommandText = @"
                        SELECT balance FROM accounts (XLOCK)
                        WHERE id = @id2
                    ";
                    cmd.Parameters.Clear();
                    cmd.Parameters.AddWithValue("@id2", toId);
                    cmd.ExecuteScalar();

                    // Perform transfer...

                    transaction.Commit();
                    return;
                }
            }
```

```
        }
        catch (SqlException ex)
        {
            if (ex.Number == 1205)  // Deadlock
            {
                retryCount++;
                if (retryCount >= maxRetries)
                    throw;

                Thread.Sleep(100 * retryCount);  // Exponential backoff
            }
            else
            {
                throw;
            }
        }
    }
}
```

Deadlock Monitoring

MySQL Monitoring

```
-- Create monitoring table
CREATE TABLE deadlock_events (
    event_id INT AUTO_INCREMENT PRIMARY KEY,
    event_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    transaction_id VARCHAR(50),
    resource_info TEXT
);

-- Monitor with trigger (simplified example)
-- In production, use application retry logic
```

MSSQL Monitoring

```sql
-- Create Extended Event session for monitoring
CREATE EVENT SESSION track_deadlocks ON SERVER
ADD EVENT sqlserver.rpc_completed,
ADD EVENT sqlserver.sql_statement_completed
ADD TARGET package0.ring_buffer(SET max_memory=2048)
WITH (
    MAX_DISPATCH_LATENCY=30 SECONDS,
    STARTUP_STATE=ON
);

-- Query current blocking chains
SELECT
    blocking_session_id,
    session_id,
    wait_type,
    wait_duration_ms
FROM sys.dm_exec_requests
WHERE blocking_session_id > 0
ORDER BY wait_duration_ms DESC;
```

Deadlock Detection Best Practices

- ✅ Enable deadlock logging/monitoring
- ✅ Implement retry logic with exponential backoff
- ✅ Monitor deadlock frequency
- ✅ Alert when deadlocks exceed threshold
- ✅ Use deadlock graph analysis (MSSQL)
- ✅ Load test to identify potential deadlocks early
- ✅ Keep deadlock resolution timeout reasonable
- ✅ Log all deadlocks for analysis

Summary Table: Lock Types Comparison

Aspect	Exclusive Lock	Shared Lock
Purpose	Prevent read and write	Prevent write only

Aspect	Exclusive Lock	Shared Lock
Compatibility	Only one transaction	Multiple transactions
Used For	INSERT, UPDATE, DELETE	SELECT with FOR SHARE
Deadlock Risk	High	Low
Concurrency	Low	High
Performance Impact	Significant	Minimal

Key Takeaways

1. **Pessimistic Locking**: Lock immediately, assume conflicts will happen
    - Use in high-contention scenarios
    - Always lock in consistent order
2. **Shared Locking**: Allow multiple readers, prevent writers
    - Maintains read consistency
    - Low deadlock risk
3. **Deadlock Prevention**: The primary strategy is consistent lock ordering
    - Keep transactions short
    - Avoid external operations while holding locks
    - Test under load
4. **Deadlock Recovery**: Implement retry logic with exponential backoff
    - Handle specific deadlock error codes
    - Limit retry attempts
    - Monitor deadlock frequency
5. **Choose Wisely**:
    - High contention → Pessimistic locking
    - Low contention → Optimistic locking (not covered, but alternative)
    - Complex workflows → Redesign to minimize locking